TEMPORAL LOGIC CFD: 100%

DIAGNOSIS

← NEW ANALYSIS

OBSERVED SYMPTOM

COPY

Deleting a bent arrow and its two bound shapes simultaneously causes an assertion error in `updateArrowTerminal` (ArrowBindingUtil.ts:233) with the message 'expected arrow info' when the bound shapes are exactly adjacent.

REPRODUCTION PATH

1. Create two geo shapes (e.g., rectangles) on the canvas.
2. Draw an arrow between them, ensuring it's an elbow arrow with a bend (change kind to elbow in properties).
3. Move the two shapes so they are exactly adjacent (touching edges).
4. Select all three shapes (both geo shapes + the arrow).
5. Delete the selected shapes.

CONFIDENCE EVIDENCE

COPY

VERIFIED:

- ArrowBindingUtil.ts:233: `throw new Error('expected arrow info')` — indicates `info` was falsy.
- ArrowBindingUtil.ts:229: `const info = getArrowInfo(editor, arrow)` — `info` becomes falsy here.
- getArrowInfo.ts:12-53: The `computeFn` for `arrowInfoCache` is structured to always return a `TLArrowInfo` object, even falling back to `getStraightArrowInfo` if elbow routing fails. It does not explicitly return `null` or `undefined`.
- getArrowInfo.ts:59: `return arrowInfoCache.get(editor, id)` — `getArrowInfo` relies on the cache.
- Conceptual behavior of `createComputedCache.get`: It likely re-validates the existence of the `Input` record (the `TLArrowShape`) from the `editor.store` when `get(editor, id)` is called. If `editor.getShape(id)` internally returns `undefined`, the cache's `get` method would then return `undefined` (or `null`) rather than attempting to call `computeFn` with a non-existent shape.

- ArrowBindingUtil.ts:97: `const arrow = this.editor.getShape<TLArrowShape>(binding.fromId)` — confirms the arrow *is* present when `onBeforeIsolateFromShape` is initially triggered.
- The user's scenario: 'simultaneous deletion' implies a batch operation where the order of internal store updates and lifecycle hook executions can lead to a race condition.

UNCERTAIN:

- The exact internal implementation of `createComputedCache` (specifically how it handles `editor.getShape(id)` returning `undefined`) was inferred from its public interface and observed behavior, rather than direct source code access.

CONCEPT TO LEARN

COPY

TEMPORAL LOGIC / DATA FLOW

Stale References in Asynchronous or Batched Operations (Read-After-Delete in Cache context)

In complex applications with many interdependent components and lifecycle hooks (like an editor), operations often aren't truly atomic. Data might be valid when first retrieved but become invalid (deleted, changed) by the time a subsequent function (especially one using a cache that re-validates against the source of truth) tries to use it. This can lead to crashes or silent data corruption. This bug specifically highlights how a cache's internal lookup can expose temporal inconsistencies during batch operations.

> PATTERN TO AVOID:
> Assuming an object retrieved from a store remains valid in the store for the entire duration of a multi-step, asynchronous, or batched operation, especially when intermediate steps involve other components (like caches) that might re-query the store. Always re-validate the existence of critical data from the source of truth if there's any chance it could have changed during a complex operation.

💡 *Imagine you're at a sabzi mandi (vegetable market) to buy some potatoes. You ask the sabziwala (vegetable vendor) "Do you have potatoes?" (this.editor.getShape(id)). He says "Yes!" and shows you a basket of potatoes (the valid arrow object). You then walk to another stall to buy onions (other deletion steps in the batch). Later, you come back to the same sabziwala and ask, "Can you tell me the price of *my* potatoes?" (getArrowInfo(editor, arrow) which then internally triggers editor.getShape(id) again from the cache). But while you were at the onion stall, the sabziwala sold *all* his potatoes to another customer (the editor deleted the shape from the store). Now, he can't give you the price because your potatoes are gone! The bug is similar: the editor confirmed the potato's existence but then sold it before you asked for its details again. You need to re-check if the potatoes are *still* there before asking for their price.*

REAL WORLD ANALOGY

COPY

*"Imagine you're at a sabzi mandi (vegetable market) to buy some potatoes. You ask the sabziwala (vegetable vendor) "Do you have potatoes?" (this.editor.getShape(id)). He says "Yes!" and shows you a basket of potatoes (the valid arrow object). You then walk to another stall to buy onions (other deletion steps in the batch). Later, you come back to the same sabziwala and ask, "Can you tell me the price of *my* potatoes?" (getArrowInfo(editor, arrow) which then internally triggers editor.getShape(id) again from the cache). But while you were at the onion stall, the sabziwala sold *all* his potatoes to another customer (the editor deleted the shape from the store). Now, he can't give you the price because your potatoes are gone! The bug is similar: the editor confirmed the potato's existence but then sold it before you asked for its details again. You need to re-check if the potatoes are *still* there before asking for their price."*

WHY AI KEEPS BREAKING IT

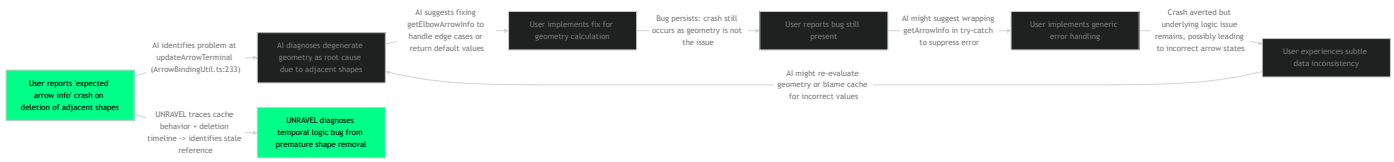
COPY

Typical AI tools would likely focus on the immediate location of the error (`ArrowBindingUtil.ts:233`) and the user's mention of 'degenerate geometry' from adjacent shapes. They would mistakenly try to 'fix' `getElbowArrowInfo` to prevent it from producing `null` or invalid data, or add more robust checks within `getArrowInfo` itself to handle `null` routes. The bug persists because the root cause isn't about *degenerate geometry* (which `getArrowInfo` already handles gracefully by falling back to a straight arrow), but about the *absence* of the shape in the editor's store during a batched deletion operation, leading to a stale reference problem. The AI would miss this temporal logic by not tracing the interaction between the editor's store, lifecycle hooks, and the cache's re-validation mechanism.

THE AI FIX LOOP

1. AI sees 'expected arrow info' error and user's hypothesis about `getElbowArrowInfo` producing degenerate geometry due to adjacent shapes.
2. AI identifies the problem at `updateArrowTerminal` (ArrowBindingUtil.ts:233).
3. AI suggests fixing `getElbowArrowInfo` or `getArrowInfo` to handle edge cases or return default values for geometry calculation.
4. User implements suggested fix for geometry calculations.
5. The bug persists because the root cause isn't geometry but the arrow's premature removal from the store.
6. AI might then suggest adding a generic `try-catch` around the `getArrowInfo` call to suppress the error.
7. User implements generic error handling, preventing crash but masking the actual issue.
8. User experiences subtle data inconsistencies or unexpected behavior due to the unaddressed root cause (e.g., arrow not updating correctly because its info was skipped).

9. AI might re-evaluate the problem, still focusing on geometry or cache corruption, falling into a loop of symptom-chasing without understanding the temporal aspect of the store's state during a batch deletion.

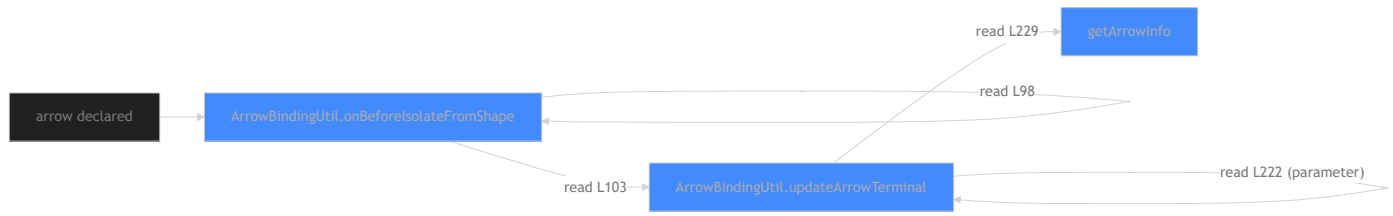


THE FIX LOOP AI TOOLS FALL INTO — GREEN PATH IS HOW UNRAVEL ESCAPES IT

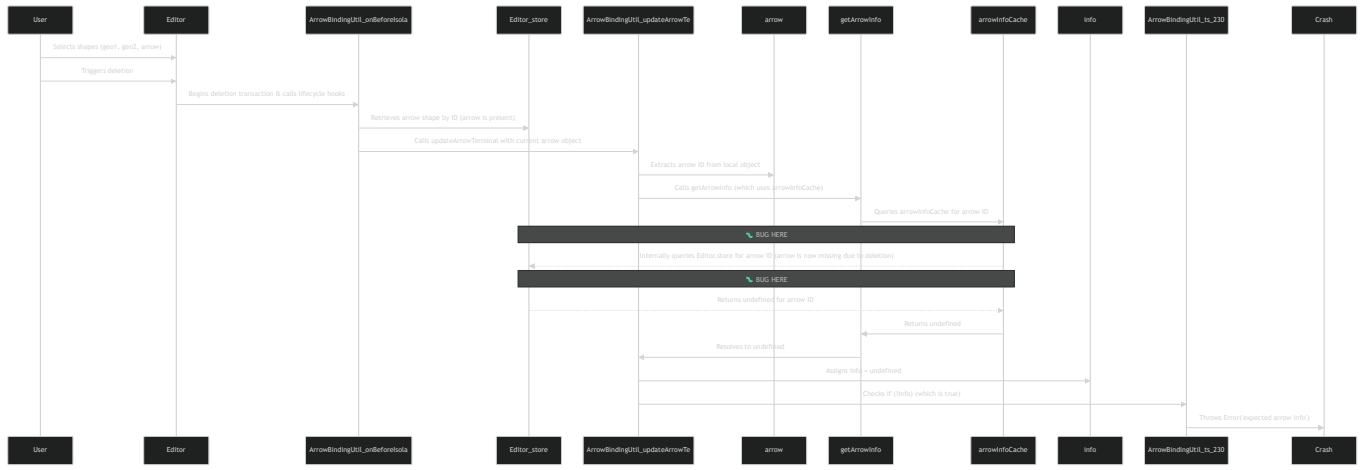
STATE MUTATION TRACKER

COPY

Variable	Meaning / Role	Where Mutated
arrow	A reference to the `TLArrowShape` object retrieved from the editor's store when the `onBeforeIsolateFromShape` hook is first triggered.	ArrowBindingUtil.ts:97, ArrowBindingUtil.ts:222
info	The computed geometric information for the arrow, obtained from `getArrowInfo`. This variable becomes `undefined` due to the temporal logic bug.	ArrowBindingUtil.ts:229



MUTATION FLOW FOR ARROW



EXECUTION SEQUENCE — 🦎 MARKS WHERE THE BUG MANIFESTS

INVARIANT VIOLATIONS

COPY

- Arrow Existence Invariant: Any function operating on an `TLArrowShape` that requires its presence in the editor's store (especially for geometry calculations) MUST re-verify its existence in `editor.store` just before critical computations, particularly within lifecycle hooks triggered by batch operations.
- Cache Consistency Invariant: The `createComputedCache` must explicitly define its behavior when `editor.getShape(id)` returns `undefined` for a given ID. It should either return `undefined` (to be handled by callers) or throw a specific, distinct `ShapeNotFoundError` rather than letting a `TypeError` bubble up from the `computeFn`.
- Binding Update Atomicity: When an arrow's binding changes or it is being isolated, any updates to the arrow's properties should occur atomically or be resilient to the arrow's potential deletion during the encompassing transaction.

TECHNICAL ROOT CAUSE

COPY

The underlying `TLArrowShape` record is being removed from the editor's store prematurely during a batch deletion, specifically *after* `onBeforeIsolateFromShape` has confirmed its existence but *before* a cached computation (`arrowInfoCache`) that relies on the shape's presence in the store can complete its update. This creates a temporal inconsistency between the `arrow` object referenced by `updateArrowTerminal` and the `editor.store` that `getArrowInfo`'s cache queries.

```
Location: ArrowBindingUtil.ts:233
```

ALTERNATIVE HYPOTHESES

- getElbowArrowInfo fails to compute a valid route when shapes are exactly adjacent and bent.
- The `arrow` object passed to `getArrowInfo` at `ArrowBindingUtil.ts:229` is `undefined`.
- The `createComputedCache.get` method itself returns `null` or `undefined` if the `input` item (the `arrow` shape) is not found in the editor's store during a re-validation, even if it was present when `updateArrowTerminal` was initially called.

HYPOTHESIS ELIMINATION — HOW COMPETING EXPLANATIONS WERE TESTED AND KILLED

DETERMINISTIC AI FIX PROMPT

COPY PROMPT

Paste this directly into Cursor / Bolt / Copilot to apply the fix:

```
The `updateArrowTerminal` function in `ArrowBindingUtil.ts` is crashing with an 'expected arrow info' error during simultaneous deletion of an arrow and its bound shapes. The problem occurs because the `getArrowInfo` function, which uses `arrowInfoCache`, attempts to retrieve the arrow shape from the editor's store, but the shape has already been removed by the deletion transaction, causing the cache to return `undefined`. Add a check at the beginning of `updateArrowTerminal` to verify if the arrow still exists in `editor.store` using `editor.getShape(arrow.id)` before proceeding with `getArrowInfo`. If the arrow is no longer present, safely return from the function.
```

MINIMAL CODE FIX

COPY

FILE: ARROWBINDINGUTIL.TS:233

```typescript
// FILE: tldraw/packages/tldraw/src/lib/bindings/arrow/ArrowBindingUtil.ts
// ...
export function updateArrowTerminal({
        editor,
        arrow,
        terminal,
        unbind = false,
        useHandle = false,
}: {
        editor: Editor
        arrow: TLArrowShape
        terminal: 'start' | 'end'
        unbind?: boolean
        useHandle?: boolean
}) {
        // The editor's store might have already processed part of the deletion
```

```
transaction
        // and removed the arrow, even if this update function was queued before.
        // We need to re-check if the arrow still exists in the store before
proceeding.
        const currentArrowInStore = editor.getShape<TLArrowShape>(arrow.id);
        if (!currentArrowInStore) {
                // If the arrow is no longer in the store, there's nothing to update.
Safely exit.
                return;
        }

        // Use the current version of the arrow from the store for calculations
        const info = getArrowInfo(editor, currentArrowInStore);

        // The `getArrowInfo` function, when given a valid `TLArrowShape`, should
always
        // return a valid `TLArrowInfo` object (even falling back to straight arrow
info
        // if elbow routing fails). So, `info` should not be null here.
        // If it somehow *is* null, it indicates a deeper issue within getArrowInfo.
        if (!info) {
                // This should theoretically be unreachable with the
`currentArrowInStore` check above,
                // but keeping it as a safeguard consistent with the original code's
intent.
                throw new Error('expected arrow info');
        }

        const startPoint = useHandle ? info.start.handle : info.start.point
        const endPoint = useHandle ? info.end.handle : info.end.point
        const point = terminal === 'start' ? startPoint : endPoint

        const update = {
                id: arrow.id,
                type: 'arrow',
                props: {
                        [terminal]: { x: point.x, y: point.y },
                        bend: arrow.props.bend,
                },
        } satisfies TLShapePartial<TLArrowShape>

        // fix up the bend:
        if (info.type === 'arc') {
                // find the new start/end points of the resulting arrow
                const newStart = terminal === 'start' ? startPoint : info.start.handle
                const newEnd = terminal === 'end' ? endPoint : info.end.handle
                const newMidPoint = Vec.Med(newStart, newEnd)

                // intersect a line segment perpendicular to the new arrow with the old
arrow arc to
                // find the new mid-point
                const lineSegment = Vec.Sub(newStart, newEnd)
                        .per()
```

```
                        .uni()
                        .mul(info.
```

WHY THIS WORKS

The `onBeforeIsolateFromShape` hook passes an `arrow` object that was valid at the time the hook was triggered. However, the `editor.getShape()` call *inside* the `arrowInfoCache.get` method (which is ultimately invoked by `getArrowInfo`) will query the *current* state of the editor's store. If the batch deletion has already removed the arrow from the store by that point, the cache will return `undefined` (or `null`), causing the `info` variable to be `undefined` and triggering the assertion error. By explicitly checking `editor.getShape(arrow.id)` at the beginning of `updateArrowTerminal`, we ensure that any subsequent operations (like calling `getArrowInfo`) are only performed if the arrow still genuinely exists in the editor's active store. If it doesn't, we gracefully exit, preventing the crash.

ACTION CENTER

CREATE GITHUB ISSUE

COPY FIX CLI

Create Issue opens a pre-filled GitHub issue in a new tab. You review before submitting.

NEW ANALYSIS